

Paesano: Holonomic Autonomous Mobile Robotics Platform

Architecture, Estimation, and Control

Joseph Marra

April 15, 2026



Figure 1: Paesano assembled hardware platform.

Contents

1	Introduction	4
2	Hardware Architecture	4
2.1	Parts List	4
2.2	System Overview	5
2.3	Power Architecture	5
2.4	Motor Control Stack	6
2.5	Sensor Suite	6
2.6	Mechanical Design	7
2.7	Design Philosophy	7
3	Firmware	8
3.1	Communication Protocol	8
3.2	Control Loop Architecture	8
3.3	Motor Control Algorithms	9
3.3.1	Velocity PID Control	9
3.3.2	Position PID Control	9
4	Software Architecture	10
5	Kinematics	11
6	Odometry	12
7	State Estimation	12
7.1	Extended Kalman Filter (EKF)	13
7.1.1	Purpose in the System	13
7.1.2	State Vector	13
7.1.3	Covariance Tuning	13
7.2	Monte Carlo Localization (MCL)	14
7.2.1	Purpose in the System	14
7.2.2	Bayesian Formulation	14

7.2.3	Motion Model	15
7.2.4	Measurement Model (Likelihood Field Model)	16
7.2.5	Resampling	18
7.3	Kidnapped Robot Problem	19
7.3.1	Limitations and Failure Modes	21
8	Navigation	21
8.1	Occupancy Grid Model	21
8.2	A^*	22
8.3	Geometric Path Post-Processing and Trajectory Synthesis	22
8.3.1	Path Pruning: String Pull	22
8.3.2	Path Smoothing and Arc-Length Parameterization	23
9	Trajectory Following	23
9.1	Linear Quadratic Regulator (LQR)	23
10	Mobile Application	25
11	Experiments	26
11.1	Velocity and Position Control Performance	26
11.2	Trajectory Following Performance	28
12	Conclusion and Future Improvements	30

1 Introduction

This technical report documents the design, implementation, and testing of Paesano, a holonomic autonomous mobile robotics platform. Paesano is intended to navigate indoor environments using probabilistic state estimation, map-based planning, and closed-loop trajectory tracking while being monitored and operated through a mobile application with live telemetry. Both the hardware and software stacks were designed to be modular so that future sensors, estimation methods, or control strategies can be integrated without redesigning the full platform.

The project was developed over two semesters. The first focused on hardware integration, power distribution, and embedded motor-control firmware. The second focused on state estimation, planning, and trajectory tracking. The following sections describe the resulting architecture, the algorithms used in the final system, and the experiments used to evaluate its performance. A central goal of the project was to gain practical experience building a full robotics stack while producing a platform that can continue to support future experimentation.

2 Hardware Architecture

2.1 Parts List

Prices shown below are approximate and may fluctuate over time based on vendor availability, shipping, and market conditions. Total estimated cost of listed components: \$644.04.

Part	Cost (USD)
HiWonder Chassis Kit	\$185.00 (approx.)
PCB Kit	\$9.99
Raspberry Pi 5 w/ Power Supply and Active Cooler	\$162.99
LiDAR	\$74.90
Buck Converter (USB-C 5V 5A)	\$9.99
22AWG Arduino Wires (Assorted)	\$6.98
Power Switch	\$7.99
Standoffs	\$7.99
Heat Shrink	\$7.99
5V Buck Converter	\$8.99
5200mAh LiPo Battery	\$35.99
LiPo Charger	\$39.98
Raspberry Pi Pico	\$14.59
XT60 Connectors	\$10.99
XT60 1-to-2 Splitter	\$12.88
BNO085	\$24.95
Motor Drivers	\$9.90
Wago Connectors	\$11.95

Additional Required Tool

- Assorted screwdrivers
- Soldering iron
- Multimeter
- Access to a 3D printer

2.2 System Overview

I designed the system around a clear separation between high-level autonomy and low-level motor control. This allows the high-level compute module to run estimation, planning, navigation, and operator-facing services without also carrying the timing burden of direct motor actuation.

The high-level compute module, a Raspberry Pi 5 running Ubuntu 24.04 LTS, handles state estimation, navigation, trajectory following, and communication with the mobile application. The motor control module, built around a Raspberry Pi Pico and two TB6612 motor drivers, manages deterministic low-level control of the drivetrain. This separation improves modularity, simplifies debugging, and makes future hardware or firmware changes less disruptive to the rest of the platform.

Figure 2 shows the resulting system architecture and the primary flow of power and information between components.

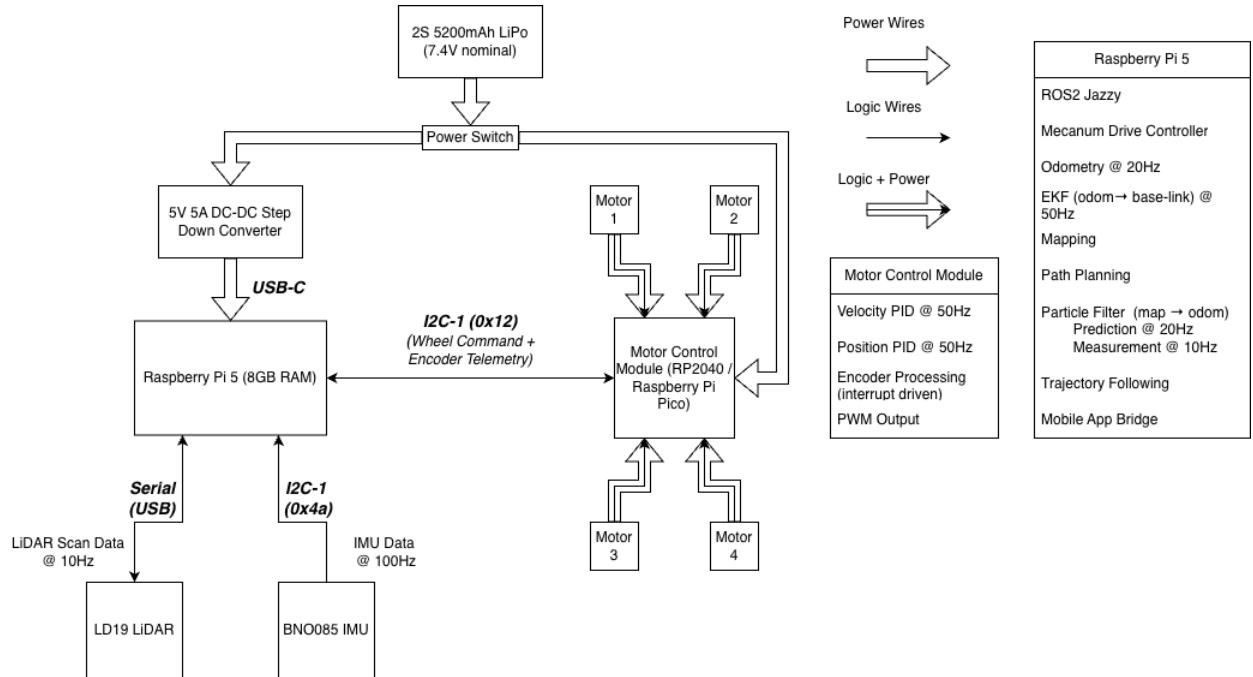


Figure 2: System architecture overview.

2.3 Power Architecture

The robot is powered by a 7.4V nominal 2S 5200mAh LiPo battery. This battery directly powers the entire system, acting as the "heart" of the robot. This battery is chosen for its high energy

density and ability to provide the necessary current for all components, especially the motor drivers. A dedicated DC-DC converter is used to step down the voltage and amperage to appropriate levels for the RPI5, ensuring stable power delivery and preventing brownouts. The motors themselves are powered directly from the 7.4V supply, since they are designed to operate at this voltage. The Pico is powered from a regulated 5V rail within the motor control module so that motor current spikes are less likely to disturb the higher-level compute hardware.

2.4 Motor Control Stack

The HiWonder chassis originally came with its own motor control module and PID controller. However, the available hardware was limited and the firmware was not practical to customize for this project. I therefore designed a dedicated motor control module built around the Pico and two TB6612 dual motor drivers.

This motor control module performs closed-loop control of the four DC motors at a fixed update rate of 50Hz. Offloading these tasks to a microcontroller keeps the low-level loop deterministic and allows the Raspberry Pi 5 to reserve its compute resources for higher-level autonomy tasks. Figure 3 shows the schematic of this subsystem.

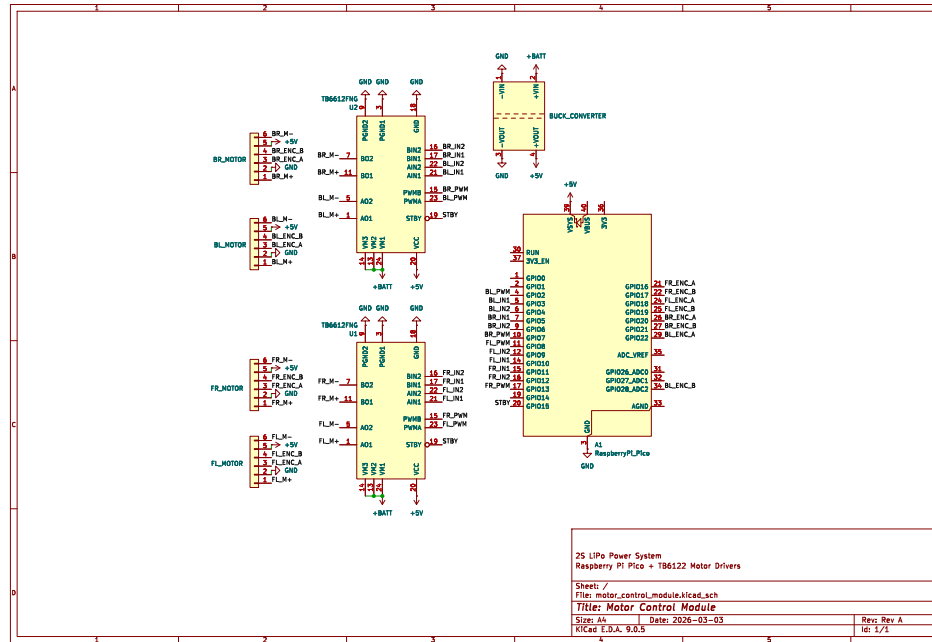


Figure 3: Motor Control Module.

2.5 Sensor Suite

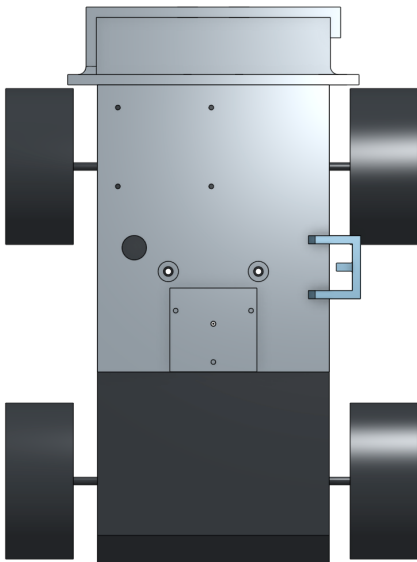
The robot uses a combination of wheel encoders, IMU, and LiDAR for perception and state estimation. A BNO085 IMU provides orientation and angular velocity measurements at approximately 100Hz. IMU data is fused with wheel odometry calculated using wheel encoders in an Extended Kalman

Filter to produce a smooth local state estimate. An LD19 2D LiDAR sensor provides planar range measurements which are used for mapping and localization. The scan data is produced at approximately 10Hz.

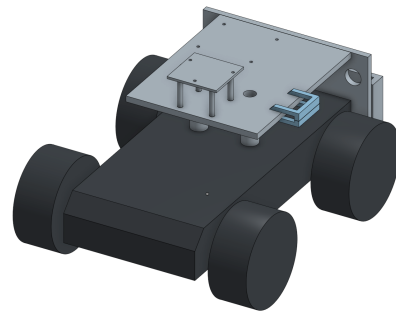
- IMU Datasheet/Documentation: <https://www.adafruit.com/product/4754>
- LiDAR Datasheet/Documentation: <https://wiki.youyeetoo.com/en/Lidar/D300>

2.6 Mechanical Design

The chassis from HiWonder provided motors with encoders and a main chassis frame. However, it did not come with any intuitive way to mount the various sensors, batteries, and other processing units. To address this, I designed several custom parts to integrate these systems into the platform. These parts were designed using OnShape and then printed on a Bambu Lab X1 Carbon in PLA.



(a) CAD Bird's Eye View



(b) CAD Isometric View

Figure 4: Mechanical CAD models of the platform and custom mounts.

The parts in gray and in blue are the ones that were custom designed and printed. The part in black is a model of the HiWonder chassis to make designing the custom parts easier. The parts in gray include a stand for the LiDAR, a mounting plate for the RPI5 and IMU, and a battery compartment in the back. The blue part is a power switch holder, and once printed it is to be super glued onto the gray mounting plate wherever desired.

2.7 Design Philosophy

The design philosophy of this robot is defined by five ideas:

- Modularity

- Simplicity
- Expandability
- Accessibility
- Cost Effectiveness

I designed the robot in a way that makes it easy to debug and iterate on. Components are physically accessible and logically separated so that individual parts of the system can be tested or replaced without affecting the entire robot. For example, low-level motor control is handled independently from high-level compute tasks, allowing each layer to be developed and debugged separately.

I also considered cost effectiveness when selecting components, favoring lower-cost parts where possible while still leaving room for future expansion. At the same time, simplicity was prioritized by avoiding unnecessary hardware and architectural complexity. The result is a platform that is practical to develop on, straightforward to modify, and capable of supporting future improvements.

3 Firmware

All firmware for this project was written in C++ for compatibility with the RP2040 microcontroller on the Pico and for predictable runtime performance. The firmware is responsible for real-time motor control, encoder processing, and communication with the Raspberry Pi 5. Because motor control requires consistent timing and low latency, this layer operates independently of the higher-level ROS2 stack.

3.1 Communication Protocol

Communication between the RPI5 and the Pico is implemented over I²C, with the RPI5 acting as the bus master and the Pico configured as an I²C slave at address 0x12. The RPI5 reads and writes register-mapped data to exchange control commands and telemetry.

Table 2: I²C register map between the RPI5 and Pico.

Name	Address	Pico Access	Purpose
I2C_ADDR	0x12	N/A	7-bit I ² C slave address for the Pico.
REG_MOTOR_SPEEDS	51	Read	Sets wheel/motor speed targets.
REG_ENCODERS	60	Write	Readback register for encoder counts.
REG_PID_GAINS	70	Read	Configuration register for PID gain parameters.

3.2 Control Loop Architecture

The main control loop of the firmware runs every 0.02s, or at 50Hz. Encoder measurements are used to estimate wheel velocities, which are fed into a cascaded control system. There are two possible modes: drive and hold.

In drive mode, the robot moves at the commanded velocity using a velocity PID with feedforward compensation. In hold mode (after the robot has been commanded to stop), a position controller generates velocity targets based on the error between the current position and the position at the time the robot was commanded to stop so it remains in place. These targets are then tracked by the velocity PID controller. The resulting PWM signals from either mode are sent to the TB6612 motor drivers.

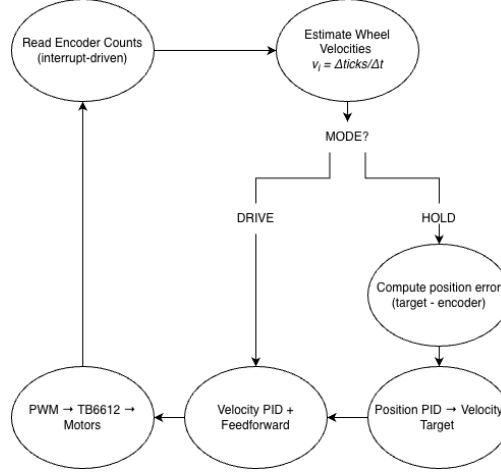


Figure 5: Control loop timing diagram.

3.3 Motor Control Algorithms

As aforementioned, motor velocity and position control is done using separate velocity and position PID controllers. Both controllers' integral term is capped when the control output saturates.

3.3.1 Velocity PID Control

$$e_v(t) = v_{\text{ref}}(t) - v(t)$$

$$u_v(t) = K_{p,v} e_v(t) + K_{i,v} \text{clip} \left(\int_0^t e_v(\tau) d\tau, -I_{\text{max},v}, I_{\text{max},v} \right) + K_{d,v} \frac{de_v(t)}{dt} + K_{\text{ff}} v_{\text{ref}}(t)$$

3.3.2 Position PID Control

$$e_p(t) = p_{\text{ref}}(t) - p(t)$$

$$v_{\text{ref}}(t) = K_{p,p} e_p(t) + K_{i,p} \text{clip} \left(\int_0^t e_p(\tau) d\tau, -I_{\text{max},p}, I_{\text{max},p} \right) + K_{d,p} \frac{de_p(t)}{dt}$$

In practice, the velocity controller handles normal motion tracking while the position controller is only engaged after a stop command. This arrangement keeps the robot responsive during motion and reduces residual drift once the commanded velocity returns to zero.

4 Software Architecture

The software in the high-level compute module, running on the Raspberry Pi 5 (RPI5), is built using the Jazzy Jalisco distribution of ROS2 on Ubuntu 24.04 LTS. ROS2 provides a modular architecture that allows complex robotic systems such as this to be composed of smaller independent nodes, communicating over standardized topics. Using ROS2 provides several advantages for this system. First, it enables clear separation between the various subsystems (state estimation, planning, control, etc.), where each system is its own node or series of nodes. Second, the publish-subscribe communication model used by ROS2 topics makes it simple to route sensor data and commands between nodes.

In this system, sensor measurements such as wheel encoder data, IMU readings, and LiDAR scans are published on dedicated topics and consumed by estimation nodes that fuse them into a consistent robot state estimate. Higher-level nodes for navigation and trajectory following use that estimate to generate motion commands, which are ultimately published as body-frame velocity commands and sent to the motor control module for actuation.

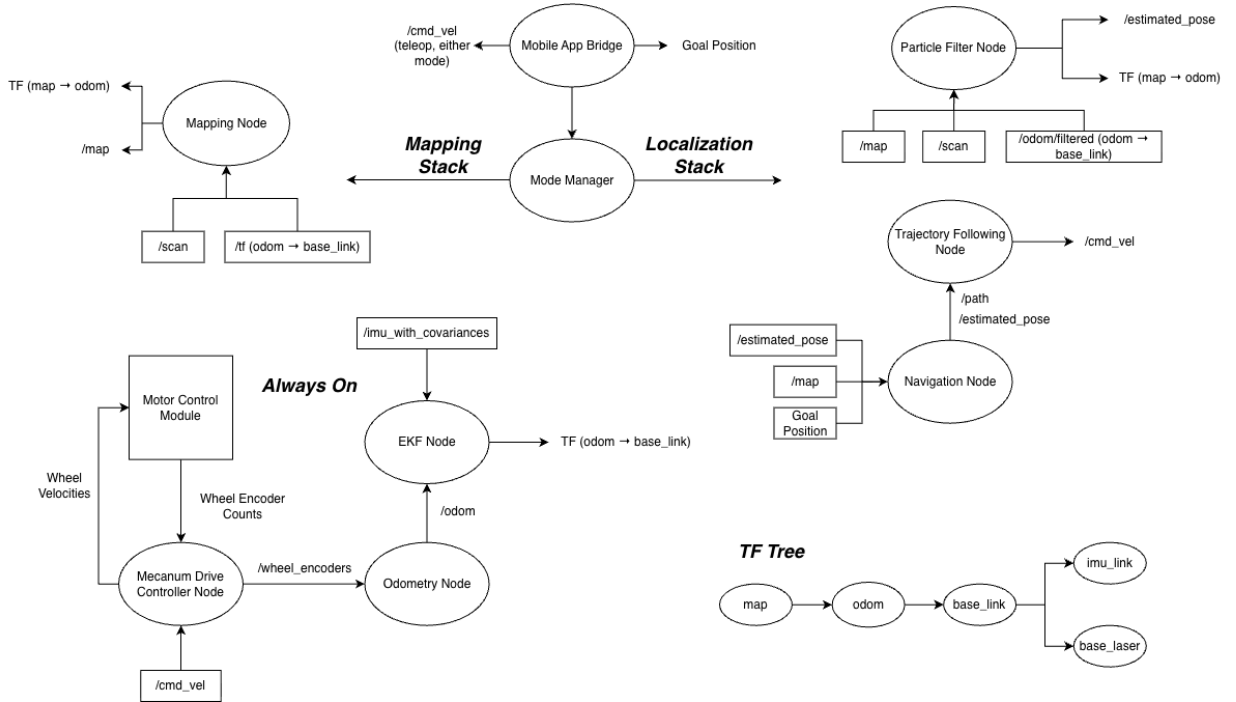


Figure 6: Software architecture overview.

The TF frame tree defines the relationships between the robot base (`base_link`), odometry frame (`odom`), map frame (`map`), and onboard sensors (`base_laser`, `imu_link`). These transforms are used by localization, navigation, and control nodes so that pose estimates and commands remain consistent across the full software stack.

5 Kinematics

To achieve holonomic motion, the robot uses a mecanum wheel drivetrain. Alternative systems such as swerve drive offer superior traction, efficiency, and high speed performance, but it also requires additional steering actuators and more complex mechanical control systems. For my purposes, the increased complexity simply isn't justifiable on a time or budget basis.

I chose mecanum drive because it provides omnidirectional movement using only four drive motors, yielding a mechanically simple but fully holonomic platform. A differential-drive robot would have been simpler, but it would not have supported lateral motion or the broader kinematic and control challenges that motivated this project. In the final real-hardware implementation, the drivetrain no longer uses a normalized wheel-mixing approximation. Instead, the high-level motor controller interprets `/cmd_vel` in SI units and applies geometry-based inverse kinematics using the measured platform dimensions:

$$r = 0.0485 \text{ m}, \quad L = 0.212 \text{ m}, \quad W = 0.194 \text{ m}, \quad R = \frac{L + W}{2} = 0.203 \text{ m}$$

For a commanded body-frame twist

$$\mathbf{u} = \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix},$$

the wheel rim linear velocities are computed as

$$\begin{bmatrix} v_{FL} \\ v_{FR} \\ v_{BL} \\ v_{BR} \end{bmatrix} = \begin{bmatrix} 1 & -1 & -R \\ 1 & 1 & R \\ 1 & 1 & -R \\ 1 & -1 & R \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}$$

where v_x and v_y are the commanded translational velocities in m/s and ω is the commanded yaw rate in rad/s.

These wheel rim velocities are then converted into encoder tick-rate targets using the known encoder resolution of $N_{enc} = 2882$ ticks per revolution. Since the linear distance traveled per encoder tick is

$$d_{\text{tick}} = \frac{2\pi r}{N_{enc}},$$

the target wheel tick rates become

$$\dot{n}_i = \frac{v_i}{d_{\text{tick}}} = v_i \frac{N_{enc}}{2\pi r}.$$

To keep the commanded motion within the measured hardware envelope, the body-frame command is first clamped to

$$\sqrt{v_x^2 + v_y^2} \leq 0.35 \text{ m/s}, \quad |\omega| \leq 1.5 \text{ rad/s}.$$

After inverse kinematics, if any wheel target exceeds the configured maximum of 3500 ticks/s, all four wheel targets are scaled by the same factor so that the requested motion direction is preserved while remaining inside the actuator limits. The final tick-rate targets are then normalized to the Pico's signed `int8` command range before being sent over I²C.

6 Odometry

Wheel odometry provides a local estimate of robot motion by integrating measured wheel velocities over time. This estimate is subject to drift due to wheel slip, friction, noise, and other factors, but it provides a high frequency state estimate that becomes useful for localization later on. It is calculated using encoder measurements from each of the four drive motors. Each one of them provides incremental encoder counts (approx. 2882 per revolution) which are used to estimate wheel angular velocity. From these measurements the robot body velocity is estimated and then integrated to produce a pose estimate in the odom frame.

Let Δn_i denote the change in encoder ticks for wheel i over a time interval Δt . Using the known ticks-per-revolution constant of the encoders, the wheel angular velocity can be computed as

$$\omega_i = \frac{2\pi}{2882} \frac{\Delta n_i}{\Delta t}$$

The velocity of the robot itself is then estimated using the mecanum wheel forward kinematics. This matrix maps the individual wheel velocities to the robot translational velocity and angular velocity.

$$\begin{bmatrix} v_{x,\text{body}} \\ v_{y,\text{body}} \\ \omega \end{bmatrix} = \begin{bmatrix} \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ -\frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{1}{4} \\ -\frac{1}{4(L+W)} & \frac{1}{4(L+W)} & -\frac{1}{4(L+W)} & \frac{1}{4(L+W)} \end{bmatrix} \begin{bmatrix} \omega_{\text{FL}} \\ \omega_{\text{FR}} \\ \omega_{\text{BL}} \\ \omega_{\text{BR}} \end{bmatrix}$$

We then convert this robot velocity into the odom frame using a rotation matrix where theta is the heading of the robot, allowing the velocity to be represented in the odom frame:

$$\begin{bmatrix} v_{x,\text{odom}} \\ v_{y,\text{odom}} \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} v_{x,\text{body}} \\ v_{y,\text{body}} \end{bmatrix}$$

”The robot pose is then updated by integrating the body velocity in the odom frame:”

$$x_{t+1} = x_t + v_{x,\text{odom}} \Delta t$$

$$y_{t+1} = y_t + v_{y,\text{odom}} \Delta t$$

$$\theta_{t+1} = \theta_t + \omega \Delta t$$

7 State Estimation

I split state estimation into local (Extended Kalman Filter) and global estimation (Monte Carlo Localization) layers. The EKF fuses wheel odometry and IMU data to produce a smooth local estimate in the odom frame, however it is subject to slip and drift. Monte Carlo Localization (MCL)

uses LiDAR and the known map to adjust for this drift. The EKF provides `odom` $\rightarrow$ `base_link` and MCL provides `map` $\rightarrow$ `odom`, giving:

$$mapT_{base_link} = mapT_{odom} odomT_{base_link}$$

7.1 Extended Kalman Filter (EKF)

For the purposes of this project, the EKF provided by the *robot_localization* package was used for simplicity of integration (https://wiki.ros.org/robot_localization).

7.1.1 Purpose in the System

The purpose of the EKF is to generate a high-frequency local state estimate (x -position, y -position, v_x , v_y , heading θ , and heading rate $\dot{\theta}$) by fusing noisy sensor data from wheel odometry and the IMU, using covariance matrices to represent the relative uncertainty of each measurement. This local estimate is used as the motion model for MCL. The underlying prediction/update equations follow the standard *robot_localization* implementation. I chose to use an EKF because it provides a reasonable balance between estimation accuracy and computational efficiency, and it handles nonlinearity in the motion model well.

7.1.2 State Vector

The EKF estimates a six-dimensional state representing the robot’s planar pose and velocity.

$$\mathbf{x} = [x, y, v_x, v_y, \theta, \dot{\theta}]^T$$

Odometry x , y , v_x , v_y , θ , $\dot{\theta}$ and IMU θ , $\dot{\theta}$ were fused to get this state. However, since odometry θ , $\dot{\theta}$ is extremely unreliable, I tuned the covariances matrices so that odometry practically owns x , y , v_x , v_y and the IMU owns θ and $\dot{\theta}$.

7.1.3 Covariance Tuning

The covariance values used for the EKF were informed by experimental odometry measurements. While the experiment was conducted in ideal conditions, the results reveal important relative characteristics of the drivetrain noise. The experiment shows that forward motion produces the lowest odometry error, while lateral (strafing) motion exhibits significantly higher uncertainty due to the inherent slip characteristics of mecanum wheels. Rotational motion introduces the largest error. Across the trials, this is the relationship between each covariance:

$$\sigma_y^2 \approx 10\sigma_x^2, \quad \sigma_{\dot{\theta}}^2 \approx 10\sigma_y^2$$

Because the experiments were conducted under ideal conditions, the resulting covariance values represent optimistic estimates of drivetrain noise. In the implementation, I only had to tune two values:



Figure 7: EKF covariance tuning reference.

a base covariance and a position vs. velocity scale factor. The base covariance value is for forward motion, and then lateral and rotational covariances were set to be 10x and 100x of that value respectively, following the experimental results. I set the position vs. velocity scale factor to be 0.5, since position is the integral of velocity, velocity has less noise amplification and is generally more accurate.

IMU covariance was informed from the datasheet noise standard deviation. It was then converted to variance, then tuned experimentally.

7.2 Monte Carlo Localization (MCL)

7.2.1 Purpose in the System

Monte Carlo Localization was chosen because it is robust to non-Gaussian uncertainty (wheel slip, LiDAR noise, sudden disturbances), multi-modal pose hypotheses (considering all possible robot poses until new data arrives to disambiguate it), and occasional odometry drift. It also works well with the LiDAR likelihood-field measurement model used in this system, and can recover from poor initial pose estimates through resampling and re-weighting.

7.2.2 Bayesian Formulation

Monte Carlo Localization is derived from the recursive Bayes filter. The belief over the robot state is updated using both the motion model (control input) and the sensor model (measurement likelihood).

$$bel(w_t) = \eta p(z_t | w_t) \int p(w_t | w_{t-1}, u_t) bel(w_{t-1}) dw_{t-1} \quad (1)$$

where

$$w_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$$

represents the robot pose at time t .

7.2.3 Motion Model

The motion model uses odometry increments to propagate each particle. First, the odometry displacement is rotated into the robot body frame:

$$\begin{bmatrix} \Delta x_{body} \\ \Delta y_{body} \end{bmatrix} = \begin{bmatrix} \cos(\theta_{odom,t-1}) & \sin(\theta_{odom,t-1}) \\ -\sin(\theta_{odom,t-1}) & \cos(\theta_{odom,t-1}) \end{bmatrix} \begin{bmatrix} \Delta x_{odom} \\ \Delta y_{odom} \end{bmatrix}$$

where $\theta_{odom,t-1}$ is the odometry heading at time $t - 1$.

Next, that body-frame displacement is rotated into the frame of the particle:

$$\begin{bmatrix} \Delta x_{part} \\ \Delta y_{part} \end{bmatrix} = \begin{bmatrix} \cos(\theta_{part,t-1}) & -\sin(\theta_{part,t-1}) \\ \sin(\theta_{part,t-1}) & \cos(\theta_{part,t-1}) \end{bmatrix} \begin{bmatrix} \Delta x_{body} \\ \Delta y_{body} \end{bmatrix}$$

The particle state is then updated as

$$\begin{aligned} x_t &= x_{t-1} + \Delta x_{part} \\ y_t &= y_{t-1} + \Delta y_{part} \\ \theta_t &= \theta_{t-1} + \Delta \theta_{odom} \end{aligned}$$

To model uncertainty in the propagated particle state, the motion model is represented as a 3D Gaussian with covariance

$$\Sigma_t = \begin{bmatrix} (x_{noise} \cdot translation)^2 & 0 & 0 \\ 0 & (y_{noise} \cdot translation)^2 & 0 \\ 0 & 0 & (\theta_{noise} \cdot rotation)^2 \end{bmatrix}.$$

Conditioned on the previous particle state and the measured odometry increment, the propagated particle state is modeled as a Gaussian random variable centered at the propagated mean pose:

$$p(w_t \mid u_t, w_{t-1}) \propto \exp \left(-\frac{1}{2} (w_t - \mu_t)^T \Sigma_t^{-1} (w_t - \mu_t) \right)$$

where

$$\mu_t = \begin{bmatrix} \bar{x}_t \\ \bar{y}_t \\ \bar{\theta}_t \end{bmatrix}$$

is the mean predicted pose.

A sampled particle state is then generated as

$$\hat{w}_t = \mu_t + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \Sigma_t).$$

7.2.4 Measurement Model (Likelihood Field Model)

A likelihood field model scores each particle by projecting the observed laser beams from that particle pose, computing the expected beam endpoints in the map, and then querying the distance from each endpoint to the nearest known obstacle in the precomputed distance field.

We represent this probability using a mixture model with Gaussian + uniform distributions:

Z_{hit} = probability that a reading was generated by a correct geometrical interaction with the map (Gaussian)

Z_{rand} = probability that a reading was generated by random noise or messy geometry (uniform distribution)

Z_{hit} and Z_{rand} must sum to 1.0.

$$P_{hit} = Z_{hit} \cdot \mathcal{N}(d_i; 0, \sigma^2)$$

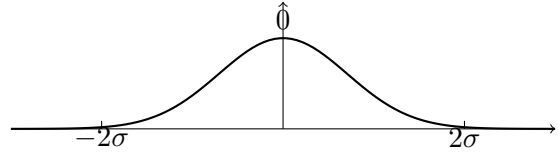


Figure 8: Gaussian likelihood model used for P_{hit} centered at the expected obstacle distance.

Z_{max} = max range of lidar

$$P_{rand} = Z_{rand} \cdot \frac{1}{Z_{max}}$$

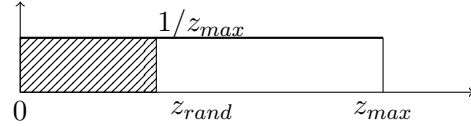


Figure 9: Uniform random measurement model used for P_{rand} .

We use a mixture model for robustness to incorrect sensor readings. For example, if a reading is 5m distance from the nearest obstacle, its Gaussian probability in log space would be:

$$\ln \left(\exp \left(\frac{-d^2}{2\sigma^2} \right) \right) = \ln(\exp(-1250))$$

In exact mathematics this value is simply -1250 . However, in floating-point computation, $\exp(-1250)$ underflows to 0, which would then force the implementation to evaluate $\ln(0)$.

$\ln(0)$ is undefined. This breaks the MCL update. By adding some probability that a reading wasn't Gaussian but part of a random uniform distribution between 0 and Z_{max} we prevent this.

So now,

$$P(z_i|x) = P_{hit} + P_{rand}$$

Here z_i denotes a single beam and x denotes the particle pose. For a full scan containing k beams, the total likelihood is computed by

$$P(Z|x) = \prod_{i=1}^k P(z_i|x)$$

However, with each $P(z_i|x)$ being less than 1, $P(Z|x)$ will underflow quickly.

To avoid this we calculate $P(Z|x)$ in log space.

$$\ln(P(Z|x)) = \sum_{i=1}^k \ln(P(z_i|x))$$

In log space, multiplication becomes addition, and summing log probabilities is numerically more stable than multiplying many probabilities less than or equal to 1.

However, for each beam the measurement model is a mixture:

$$P(z_i | x) = P_{hit} + P_{rand}.$$

This means the per-beam log likelihood cannot be written as the sum of the individual log terms. Instead, we must evaluate

$$\ln(P_{hit} + P_{rand})$$

directly.

To do this robustly, the logarithms of the two mixture components are computed separately, and the result is combined using the log-sum-exp identity. This preserves the mixture structure while avoiding numerical underflow during scan likelihood accumulation.

Let

$$a = \ln(P_{hit})$$

$$b = \ln(P_{rand})$$

$$\ln(P_{hit} + P_{rand}) = \ln(e^a + e^b) = m + \ln(e^{a-m} + e^{b-m})$$

where

$$m = \max(a, b)$$

For each particle, the total scan log likelihood is computed as

$$\log P(Z | x) = \sum_{i=1}^k \left[m_i + \ln(e^{a_i - m_i} + e^{b_i - m_i}) \right],$$

where

$$m_i = \max(a_i, b_i).$$

This produces an unnormalized scan log likelihood for each particle. In the implementation, that likelihood is applied multiplicatively to the particle's existing weight and then the result is normalized.

Let

$$c = \max_i \log w_i$$

where $\log w_i$ is the scan log likelihood of particle i .

Then define the shifted updated weights

$$\tilde{w}_i = w_{i,t-1} \exp(\log w_i - c).$$

Subtracting the maximum log weight ensures that the largest exponent is zero, which prevents numerical overflow.

The normalized particle weights are then

$$w_i = \frac{\tilde{w}_i}{\sum_j \tilde{w}_j},$$

so that

$$\sum_i w_i = 1.$$

7.2.5 Resampling

Resampling particles is necessary because it reduces the problem of degeneracy where most particles get near-zero weight. This process:

- removes low-weight particles
- duplicates high-weight particles
- keeps the particle set focused on likely states
- improves estimation quality for the same compute budget

Resampling is triggered by calculating the effective sample size of all the particles, which in this implementation means how many particles are contributing meaningful weight to the state estimate. The equation for this is:

$$N_{eff} = \frac{1}{\sum_{i=1}^N w_i^2}$$

Here N is the total number of particles and w_i is the weight of each particle. An effective sample size closer to N means that most of the particles contribute meaningfully to the state estimate. An effective sample size closer to 1 means only a few particles dominate the estimate. We resample the

particles when $N_{eff} < 0.5 * N$.

In this implementation, systematic resampling is done with a cumulative distribution (CDF) over normalized weights. This method achieves the four goals listed above. It removes low-weight particles because a low-weight particle will have a low interval in the CDF, making it less likely to be selected. It duplicates high-weight particles because a high-weight particle has a high interval in the CDF, making it more likely to be selected. Because it duplicates high weight particles it keeps the particle set focused on likely states, and only focusing on likely states improves estimation quality for the same compute budget. For a 10-particle example:

$$\begin{aligned}\mathbf{w} &= [0.05, 0.10, 0.02, 0.18, 0.08, 0.07, 0.20, 0.06, 0.14, 0.10] \\ \mathbf{CDF} &= [0.05, 0.15, 0.17, 0.35, 0.43, 0.50, 0.70, 0.76, 0.90, 1.00] \\ N &= 10, \quad \text{step} = \frac{1}{N} = 0.1, \quad r \sim \mathcal{U}\left(0, \frac{1}{N}\right)\end{aligned}$$

For this example, we use $r = 0.04$

This random offset r prevents the algorithm from repeatedly hitting the same CDF intervals. The systematic samples are:

$$u_i = r + (i - 1) \cdot \text{step} \Rightarrow [0.04, 0.14, 0.24, 0.34, 0.44, 0.54, 0.64, 0.74, 0.84, 0.94]$$

For each u_i , pick the first CDF bin where $C_j \geq u_i$:

$$[0, 1, 3, 3, 5, 6, 6, 7, 8, 9]$$

These are the source particle indices copied into the resampled set, then all new weights are reset to $1/N$.

After resampling, we then add some Gaussian noise to the state of each particle, since the CDF may cause many particles to be exactly the same, causing particle impoverishment. Adding this noise spreads duplicated particles slightly and restores local diversity.

7.3 Kidnapped Robot Problem

To address the kidnapped robot problem, an adaptive random particle injection strategy was implemented

Given a set of particle log-likelihoods $\ell_i = \log p(z \mid x_i)$, the mean likelihood across particles is computed in a numerically stable manner using the log-sum-exp trick:

$$\log \sum_i e^{\ell_i} = \ell_{\max} + \log \sum_i e^{\ell_i - \ell_{\max}}$$

The average likelihood is then:

$$\log \bar{p} = \log \left(\frac{1}{N} \sum_i e^{\ell_i} \right) = \ell_{\max} + \log \sum_i e^{\ell_i - \ell_{\max}} - \log N$$

To normalize for the number of laser beams B , a scan quality metric is defined as:

$$q = \exp\left(\frac{\log \bar{p}}{B}\right)$$

Two exponential moving averages are maintained:

$$\begin{aligned} w_{\text{fast}} &\leftarrow w_{\text{fast}} + \alpha_{\text{fast}}(q - w_{\text{fast}}) \\ w_{\text{slow}} &\leftarrow w_{\text{slow}} + \alpha_{\text{slow}}(q - w_{\text{slow}}) \end{aligned}$$

The ratio between these averages is used to detect localization degradation:

$$r = \frac{w_{\text{fast}}}{w_{\text{slow}}}$$

Using the ratio of fast and slow averages is more robust than reacting to the instantaneous confidence alone, since scan quality can fluctuate momentarily due to occlusion, dynamic obstacles, or local map ambiguity. The two time scales therefore detect a sustained drop relative to the recent baseline, which is a better indicator of the kidnapped robot condition.

An adaptive injection probability is then defined as:

$$p_{\text{adapt}} = \text{clamp}(1 - r, 0, 1)$$

This value is used to interpolate between a baseline and maximum random particle percentage:

$$\begin{aligned} \beta &= \text{clamp}(k \cdot p_{\text{adapt}}, 0, 1) \\ P_{\text{random}} &= P_{\text{base}} + (P_{\text{max}} - P_{\text{base}})\beta \end{aligned}$$

The number of injected particles is:

$$N_{\text{inject}} = \frac{P_{\text{random}}}{100} \cdot N$$

Injected particles are sampled by choosing random free cells from the map and then perturbing each selected cell uniformly within one map resolution:

$$x \sim \mathcal{U}(x_{\text{cell}} - r, x_{\text{cell}} + r), \quad y \sim \mathcal{U}(y_{\text{cell}} - r, y_{\text{cell}} + r), \quad \theta \sim \mathcal{U}(-\pi, \pi)$$

To improve robustness, adaptive random injection is only enabled when the robot is not navigating, or when its yaw rate remains below the rapid-turn threshold:

$$\text{allow} = (|\dot{\theta}| \leq \tau) \vee (\neg \text{navigating})$$

This prevents spurious particle dispersion caused by scan distortion or intentional motion.

7.3.1 Limitations and Failure Modes

There are a few situations in which the particle filter and adaptive injection strategy may fail.

- If the robot is navigating and experiences a sudden large disturbance (e.g. colliding with an obstacle), the scan quality may drop significantly while the robot is still moving, which could trigger random particle injection at a time when the filter is already struggling to maintain a coherent pose estimate.
- If the environment contains large ambiguous areas (e.g. long corridors or open spaces) where many poses produce similar scan likelihoods, the particle filter may struggle to converge to a single pose and may require more particles or additional sensor modalities for disambiguation.
- If the map is inaccurate or outdated, the measurement model may produce consistently low likelihoods even when the robot is correctly localized, leading to excessive random particle injection and potential divergence of the filter.

Tuning the particle filter parameters, such as the number of particles, noise, and various likelihood field model parameters, helped mitigate these failure modes. Under extreme circumstance, the filter may still fail, but it is robust enough to handle typical navigation scenarios and to recover from temporary localization degradation.

8 Navigation

8.1 Occupancy Grid Model

Global planning is performed on a 2D occupancy grid, but the planner does not treat all free cells equally. One thing I noticed is that if I run A* on a raw occupancy grid, the path hugs walls and creates a path impossible for the robot to follow. Rather than running A* directly on the raw binary map, the implementation first constructs an **inflated clearance-aware cost map**. This allows the robot to prefer routes that maintain additional distance from obstacles instead of simply finding the shortest geometric path through free space.

The process begins by computing, for every grid cell, the distance to the nearest occupied or unknown cell. Occupied cells and unknown cells are both treated as obstacle sources during this step. A Dijkstra-style expansion over the 8-connected grid is then used to propagate the nearest-obstacle distance throughout the map, producing a clearance field

$$d(\mathbf{c}) = \text{distance from cell } \mathbf{c} \text{ to the nearest obstacle.}$$

A hard safety buffer is then enforced around obstacles. If a cell lies within the configured obstacle buffer distance d_b , it is treated as non-traversable:

$$d(\mathbf{c}) \leq d_b \implies \mathbf{c} \text{ is blocked.}$$

For cells outside this buffer, the planner assigns a traversal penalty that decreases as clearance increases. In the current implementation this penalty is not binary and not purely geometric; it is a smooth **exponential** cost applied on top of the normal grid step cost:

$$J_{\text{clear}}(\mathbf{c}) = \lambda \exp\left(-\frac{d(\mathbf{c}) - d_b}{\ell}\right), \quad d(\mathbf{c}) > d_b$$

where λ is the clearance cost scale and ℓ is the clearance decay length. This means that cells just outside the obstacle buffer still carry a significant cost, while cells farther from obstacles rapidly become inexpensive. As a result, the planner naturally biases paths toward wider, safer corridors without discarding valid narrow passages when they are necessary.

During search, this clearance penalty is added directly to the normal A* step cost for each candidate neighbor. Axis-aligned steps retain a nominal cost of 1, diagonal steps retain a nominal cost of $\sqrt{2}$, and the obstacle-clearance penalty is added to both. The resulting planner is therefore still shortest-path based, but with a traversal model that explicitly favors obstacle clearance rather than minimizing Euclidean length alone.

8.2 A*

Global path planning is performed using the A* algorithm on a 2D occupancy grid. Since the map is discretized into square cells, the raw output of A* is a sequence of adjacent grid coordinates rather than a continuous trajectory. This makes the initial path safe and graph-search optimal with respect to the grid cost model, but it also makes the output inherently jagged. Such a path is not ideal for a physical robot, especially one that will later be tracked by a continuous controller.

For this reason, the global planner is treated as the first stage of a path generation pipeline rather than the final trajectory itself. The A* search produces a collision-free sequence of cells, after which geometric post-processing is applied to reduce unnecessary waypoints and convert the discrete path into a smoother trajectory suitable for tracking.

8.3 Geometric Path Post-Processing and Trajectory Synthesis

8.3.1 Path Pruning: String Pull

The first post-processing step is a String Pulling algorithm. Its purpose is to prune redundant intermediate nodes introduced by the grid representation. Rather than forcing the robot to visit every cell transition selected by A*, the string-pull stage searches for the longest obstacle-free straight-line connections between waypoints.

This reduces the path to the minimum number of straight-line segments required to traverse the environment without colliding with obstacles. In practice, this removes much of the staircase-like structure produced by the grid and produces a cleaner geometric path before any smoothing is attempted.

8.3.2 Path Smoothing and Arc-Length Parameterization

Following string pulling, the path consists of sparse, unevenly spaced waypoints. To produce a reference trajectory suitable for the LQR controller, we apply a **Centripetal Catmull-Rom spline** followed by a uniform resampling stage.

Spline Selection The centripetal parameterization ($\alpha = 0.5$) was chosen over the standard uniform version because it inherently handles the non-uniform waypoint spacing produced by the A^* and string pulling stages. This choice prevents the "overshoot" and unrealistic loops often seen in uniform splines when waypoints are clustered. Furthermore, as an interpolating spline, it ensures the robot passes exactly through the meaningful geometric constraints of the pruned path.

The Parameterization Problem Standard spline evaluation uses a relative parameter $t \in [0, 1]$ for each segment. However, t does not map linearly to physical distance. Sampling at fixed increments of Δt causes the trajectory to become "stretchy": long segments are sampled sparsely, while short segments are sampled densely. This creates a fluctuating geometric error signal for the LQR controller, leading to inconsistent velocity profiles and "jittery" tracking.

Uniform Resampling Implementation To ensure a consistent reference signal, the spline is re-parameterized by arc length s through a three-stage numerical process:

1. **Dense Modeling:** Each spline segment is evaluated at high resolution (100 samples per segment) to create a dense polyline approximation of the curve, denoted as points $\mathbf{q}_0, \mathbf{q}_1, \dots, \mathbf{q}_N$.
2. **Cumulative Distance:** The discrete cumulative arc length ℓ is computed for every dense point:

$$\ell_0 = 0, \quad \ell_k = \ell_{k-1} + \|\mathbf{q}_k - \mathbf{q}_{k-1}\|$$

3. **Linear Resampling:** The final trajectory points $\hat{\mathbf{p}}_m$ are generated at uniform spatial intervals of $\Delta s = 0.05$ m. For a target distance $s_m = m \cdot \Delta s$, we locate the interval $\ell_k \leq s_m \leq \ell_{k+1}$ and interpolate:

$$\hat{\mathbf{p}}_m = (1 - \lambda_m)\mathbf{q}_k + \lambda_m\mathbf{q}_{k+1}, \quad \text{where} \quad \lambda_m = \frac{s_m - \ell_k}{\ell_{k+1} - \ell_k}$$

This ensures that the controller receives waypoints at a constant spatial frequency, allowing for stable gain application and smooth motion regardless of the original path geometry.

9 Trajectory Following

9.1 Linear Quadratic Regulator (LQR)

Trajectory following is performed using a Linear Quadratic Regulator (LQR). Since the platform is a holonomic Mecanum robot operating at relatively low speed, a discrete-time kinematic state-space

model is sufficient for the controller design. The state is chosen as the planar robot pose

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix}$$

and the control input is chosen as the commanded body velocities

$$\mathbf{u}_k = \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}.$$

The discrete kinematic update for each state component is

$$x_{k+1} = x_k + \dot{x} \Delta t, \quad y_{k+1} = y_k + \dot{y} \Delta t, \quad \theta_{k+1} = \theta_k + \dot{\theta} \Delta t$$

which can be written in matrix form as

$$\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k + \mathbf{B}\mathbf{u}_k$$

with

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \mathbf{I}, \quad \mathbf{B} = \Delta t \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \Delta t \mathbf{I}.$$

Objective Function and Value Function The controller is designed to minimize the infinite-horizon quadratic cost J , which serves as the cumulative penalty for state error and control effort over an infinite timeline:

$$J = \sum_{k=0}^{\infty} (\mathbf{x}_k^T \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^T \mathbf{R} \mathbf{u}_k)$$

While J defines the optimization goal, the actual minimum achievable cost from a given state is described by the **Value Function**, $V(\mathbf{x}_k) = J_{\min}$. For a linear-quadratic regulator, this value represents the total "cost-to-go" assuming an optimal policy is followed from the current time k onward. Because the optimal control is a deterministic function of the state, the control input $\mathbf{u}$ is eliminated from the final expression, leaving $J_{\min}$ as a function of the error $\mathbf{x}_k$:

$$J_{\min} = \mathbf{x}_k^T \mathbf{P} \mathbf{x}_k$$

Solving the DARE The matrix $\mathbf{P}$ is the steady-state solution to the **Discrete Algebraic Riccati Equation (DARE)**:

$$\mathbf{P} = \mathbf{A}^T \mathbf{P} \mathbf{A} - \mathbf{A}^T \mathbf{P} \mathbf{B} (\mathbf{R} + \mathbf{B}^T \mathbf{P} \mathbf{B})^{-1} \mathbf{B}^T \mathbf{P} \mathbf{A} + \mathbf{Q}$$

In this framework, $\mathbf{P}$ acts as a compressed representation of the system's future behavior. It encodes the dynamics $(\mathbf{A}, \mathbf{B})$ and the weights $(\mathbf{Q}, \mathbf{R})$ into a constant operator that maps any current pose error into a scalar value representing the total future penalty.

Control Synthesis Once $\mathbf{P}$ is determined, the optimal feedback gain $\mathbf{K}$ is synthesized to minimize the sum of the immediate step cost and the resulting future cost:

$$\mathbf{K} = (\mathbf{R} + \mathbf{B}^T \mathbf{P} \mathbf{B})^{-1} \mathbf{B}^T \mathbf{P} \mathbf{A}$$

The final control law combines this feedback with a feedforward term ($\mathbf{u}_{ff}$) derived from the arc-length parameterized reference trajectory:

$$\mathbf{u}_{total} = \mathbf{K} \mathbf{e} + \mathbf{u}_{ff}$$

The feedback term drives the robot back toward the spline trajectory points, while the feedforward term provides the nominal velocity required to traverse the path. This ensures that the robot does not merely react to errors but actively tracks the geometry of the intended path.

10 Mobile Application

Paesano is operated through a custom iOS application that serves as the primary user interface for robot interaction. Rather than relying on terminal commands or ad hoc ROS tooling during normal use, the app exposes the robot’s current state, map data, and navigation controls in a single interface designed for real-time operation.

The application connects directly to the robot over the local network using a manually entered IP address or hostname. Once connected, it uses HTTP endpoints for discrete actions such as mode switching, goal submission, map saving, and navigation pause or cancel requests. A WebSocket connection is used in parallel to stream state updates and map revisions back to the phone so the interface can update continuously without requiring constant manual refresh.

The mobile interface is organized around the robot’s high-level operating modes:

- **Mapping mode:** Provides joystick-based teleoperation for manual exploration while showing the occupancy map live. The user can save the resulting map directly from the application once sufficient coverage has been achieved.
- **Navigation mode:** Displays the current occupancy map, estimated robot pose, and planned path. The user can tap directly on the map to submit a navigation goal, then pause, resume, or cancel the path while observing the live telemetry response.

This interface effectively acts as the operator console for Paesano. It bridges the autonomy stack and the human operator by combining map visualization, mode management, manual override, and telemetry feedback into a single control surface.

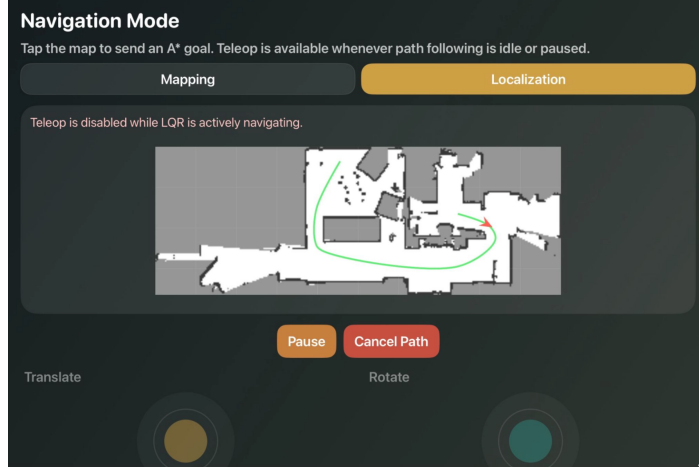


Figure 10: Paesano mobile application interface for telemetry, teleoperation, and navigation control.

11 Experiments

To evaluate Paesano at both the low-level control and high-level autonomy levels, I conducted two experiments. The first isolates drivetrain command tracking and stop behavior. The second evaluates the full autonomy stack by measuring how closely the robot follows planned trajectories in the map frame. Together, these experiments validate the platform from motor control through navigation execution.

11.1 Velocity and Position Control Performance

For the low-level velocity and position control tests, ROS2 bags were recorded while forward, lateral, and rotational commands were applied over multiple different magnitudes. The bags included `/cmd_vel`, `/odom`, `/odom/filtered`, and `/wheel_encoders`. From these logs, rise behavior, overshoot, settling, and residual stop error were observed. When a historical test bag contained a command larger than the final enforced controller limit, the analysis treated that command as the corresponding effective cap of 0.35 m/s for translation or 1.5 rad/s for rotation.

Table 3: Average low-level control metrics across the recorded velocity-control trials.

Motion	Mean steady-state error	Mean overshoot	Mean stop decay	Mean encoder stop error
Forward	−0.0067 m/s	5.79 %	0.35 s	0.0097 m
Lateral	−0.0056 m/s	5.66 %	0.25 s	0.0073 m
Rotation	−0.0245 rad/s	11.06 %	0.33 s	N/A

The forward and lateral responses were consistent across trials, with small overshoot and short residual settling after the command returned to zero. Forward tracking was the cleanest overall, with a mean steady-state error of −0.0067 m/s, indicating that the measured forward speed settled only slightly above the effective command. Lateral motion remained similarly well controlled, with

a mean steady-state error of -0.0056 m/s, although its stop-position behavior still reflected the higher slip sensitivity of mecanum strafing. Rotational tracking also settled quickly, with a mean steady-state error of -0.0245 rad/s. This negative sign again indicates mild overtracking rather than underspeed. Even so, the post-stop rotational decay remained short and consistent, which is the more important behavior for the higher-level navigation stack.

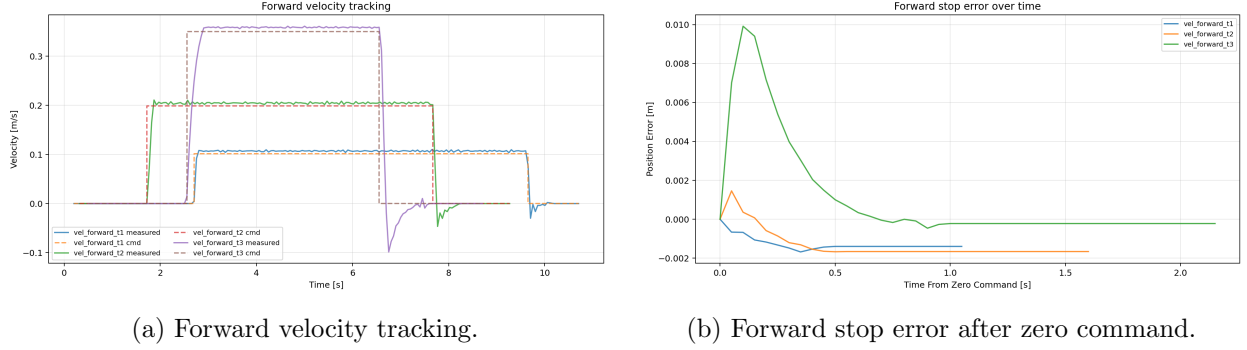


Figure 11: Forward-axis low-level control performance. The measured response closely tracks the commanded input, with only a brief braking transient when the command returns to zero.

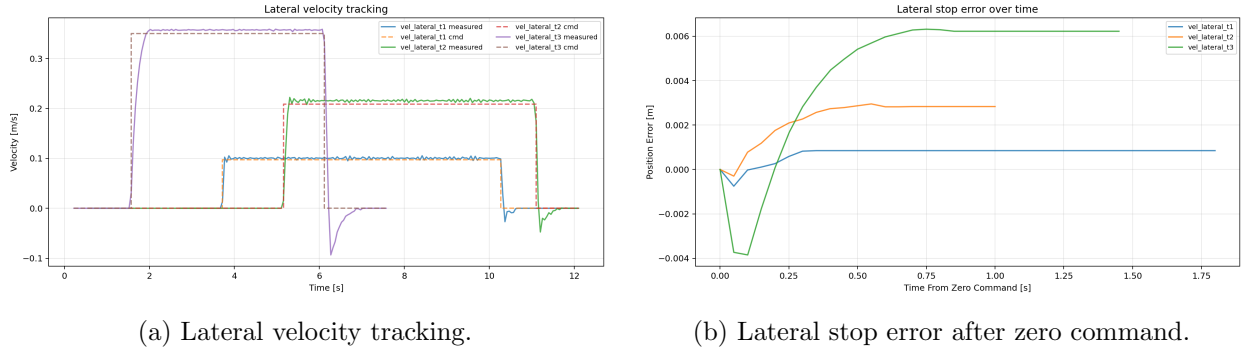


Figure 12: Lateral-axis low-level control performance. Strafing remained stable and repeatable, although slightly less precise than the forward axis.

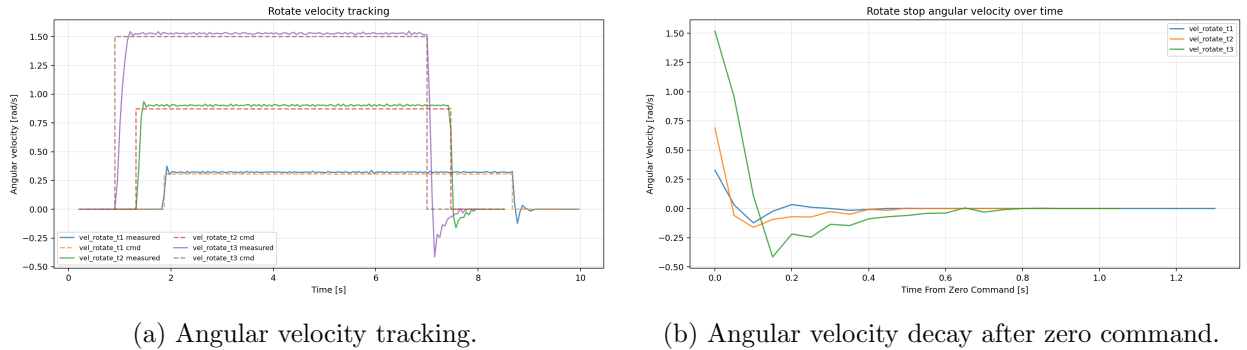


Figure 13: Rotational low-level control performance. The turn response exhibits slightly higher transient error at large commands, but the stop behavior remains sharp and repeatable.

11.2 Trajectory Following Performance

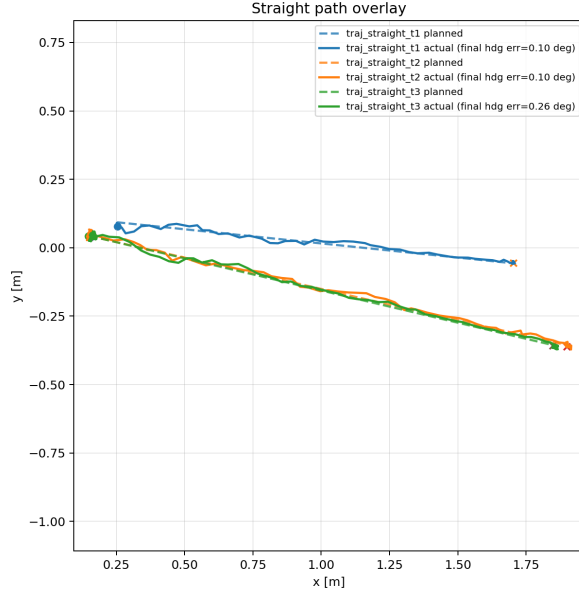
The second experiment evaluated the full autonomy stack in localization mode. Three routes were tested: a straight route, a single-turn curve route, and a longer multi-segment route through a more cluttered portion of the map. Each route was executed three times from a fixed marked start pose, and ROS 2 bags were recorded containing `/path`, `/estimated_pose`, `/odom/filtered`, `/cmd_vel`, and `/map`. The primary metrics were completion time, final position error, final heading error, and cross-track error with respect to the planned path.

This experiment intentionally validates more than just the tracking controller. Because the robot is operating in the map frame while using the full navigation stack, the results reflect the combined behavior of localization, global planning, path pruning, spline generation, and LQR tracking. A separate standalone localization benchmark was therefore not included in the final report, since route-following accuracy already depends on the pose estimator being correct over the full run.

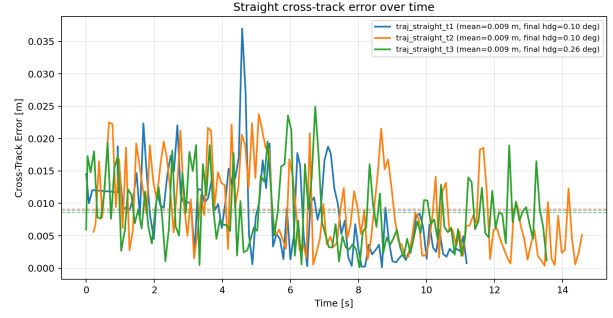
Table 4: Average trajectory-following results by route type.

Route	Mean completion time	Mean final position error	Mean cross-track error	Mean final heading error	Success
Straight	6.11 s	0.0041 m	0.0089 m	0.154 deg	3/3
Curve	18.29 s	0.0041 m	0.0086 m	0.366 deg	3/3
Complex	44.09 s	0.0095 m	0.0087 m	0.270 deg	3/3

Across all nine recorded navigation runs, the mean cross-track error was 0.0088 m, the mean final position error was 0.0059 m, and every trial completed successfully. These values are strong for a student-built indoor mobile platform and indicate that the path planner, spline generator, and LQR tracker are all operating consistently in the same frame as the localization estimate. The straight route produced the smallest directional variation, while the curve and complex routes remained nearly identical in mean cross-track error despite the larger path length and longer execution time.

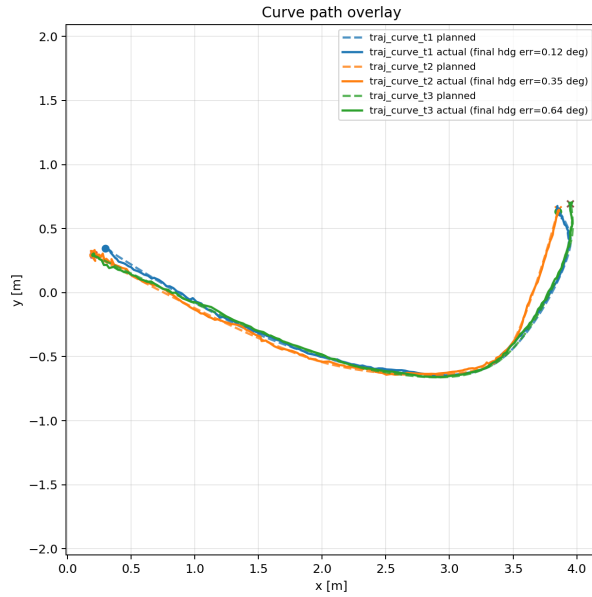


(a) Straight-route planned and actual paths.

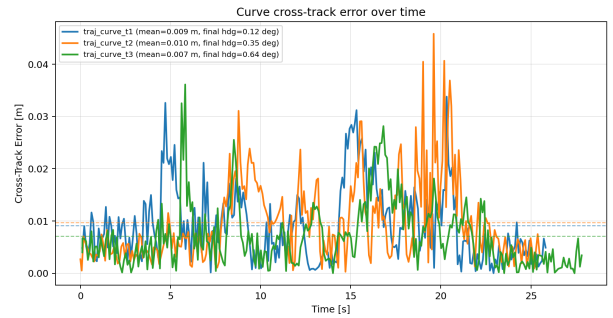


(b) Straight-route cross-track error over time.

Figure 14: Trajectory-following results for the straight route. The actual path remains nearly coincident with the planned reference, and the cross-track error stays small throughout the run.

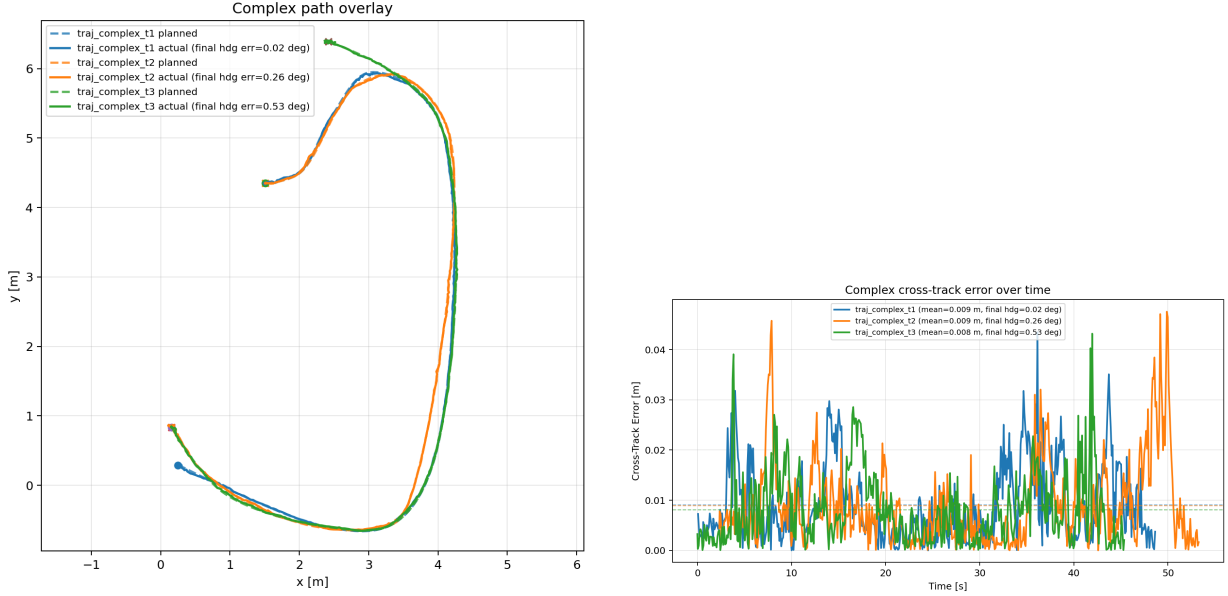


(a) Curve-route planned and actual paths.



(b) Curve-route cross-track error over time.

Figure 15: Trajectory-following results for the single-turn curve route. The controller remains tightly aligned with the planned reference through the turn.



(a) Complex-route planned and actual paths.

(b) Complex-route cross-track error over time.

Figure 16: Trajectory-following results for the complex multi-segment route. Even on the longest path, the measured tracking error remains small relative to the map scale.

12 Conclusion and Future Improvements

Paesano demonstrates that a modest-cost holonomic platform can support a complete autonomy stack spanning embedded motor control, probabilistic state estimation, map-based planning, trajectory generation, and operator interaction through a mobile application. Architecturally, the project separates deterministic low-level actuation from higher-level computation on the Raspberry Pi 5, which simplified development and made the system easier to debug and extend.

The experimental results support that architecture. At the low level, the drivetrain exhibited repeatable command tracking with small steady-state error, modest overshoot, and short post-stop decay across forward, lateral, and rotational motion. At the system level, the robot followed planned map-frame trajectories with a mean cross-track error of 0.0088 m, a mean final position error of 0.0059 m, and successful completion of all recorded navigation runs. These results indicate that the estimation, planning, and control layers are operating consistently within the same frame and within the practical limits of the hardware.

The most important future improvements are now incremental rather than architectural. On the hardware side, the platform would benefit from a higher quality LiDAR, a more rigid mechanical structure, and more complete electrical packaging. On the software side, the most promising extensions are improved localization robustness in ambiguous environments, richer navigation behaviors, and broader benchmarking under more varied conditions. Because the current design is modular at both the firmware and ROS2 levels, those future improvements can be integrated without restructuring the full system.